

Unit III Notes: Monte Carlo Methods (Prediction, Control, and Exploring Starts)

May 28, 2026

Notation

- S_t — state observed at time step t ; A_t — action selected at time step t ; R_{t+1} — reward received after taking A_t in S_t .
- An *episode* is one complete trajectory $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_T$ ending in a terminal state S_T .
- $\gamma \in [0, 1]$ — the discount factor (how much future rewards are valued compared with the next one).
- G_t — the *return* from step t , the discounted sum of all rewards received from step $t + 1$ until termination:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + \gamma^{T-t-1} R_T.$$

It is computed backwards from the terminal step using $G_t = R_{t+1} + \gamma G_{t+1}$, with $G_T = 0$.

- π — a *policy*, that is, a rule that maps each state to a probability distribution over actions; $\pi(a | s)$ is the probability of choosing action a in state s .
- $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ — the state-value function under policy π (the expected return when starting in s and following π afterwards).
- $q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ — the action-value function under π (the expected return starting from s , taking a , then following π).
- $V(s)$ and $Q(s, a)$ — the current estimates of $v_\pi(s)$ and $q_\pi(s, a)$ maintained by the algorithm.
- π_* — an optimal policy; $v_*(s)$ and $q_*(s, a)$ — the corresponding optimal value functions.
- A *visit* to state s is any time step t within an episode for which $S_t = s$; a *first visit* is the earliest such t in that episode. A visit to the pair (s, a) is any t with $S_t = s$ and $A_t = a$.
- ε — a small positive constant that fixes how often a soft policy explores a non-greedy action.
- *Stochastic* means *random*; *deterministic* means *fixed, with no randomness*. Both words appear repeatedly in the notes.

Part A — Short Questions (2 marks each)

1. Distinguish between the prediction problem and the control problem in reinforcement learning.

The *prediction* problem fixes a policy π and asks for the value function $v_\pi(s)$ or $q_\pi(s, a)$, that is, an honest evaluation of how good the given policy is. The *control* problem instead asks for an optimal policy π_* along with its value function. Prediction is therefore an evaluation

task with a known target policy, while control is an optimisation task whose answer is a new policy that achieves the largest possible expected return.

- 2. Numerical — Returns and first-visit / every-visit estimates.** In a single episode the agent visits state s at $t = 0$ and $t = 2$, receives rewards $R_1 = 4$, $R_2 = 0$, $R_3 = 8$, and terminates. Using $\gamma = 0.5$, compute the returns G_0 , G_1 , G_2 , and report the estimate $V(s)$ from this single episode under (i) first-visit Monte Carlo and (ii) every-visit Monte Carlo.

Working backwards with $G_t = R_{t+1} + \gamma G_{t+1}$ and $G_3 = 0$:

$$G_2 = 8 + 0.5(0) = 8, \quad G_1 = 0 + 0.5(8) = 4, \quad G_0 = 4 + 0.5(4) = 6.$$

First-visit Monte Carlo uses only the return from the first visit to s , namely $G_0 = 6$, so $V(s) = 6$. Every-visit Monte Carlo averages the returns from both visits, giving $V(s) = (G_0 + G_2)/2 = (6 + 8)/2 = 7$. The two estimates differ on a single episode but converge to the same limit as the number of episodes grows.

- 3. State the core idea of Monte Carlo prediction and the assumption it places on the task.**

Monte Carlo prediction estimates $v_\pi(s)$ by averaging the returns observed from many sample episodes that start (or pass through) state s . It uses no model of the environment — only sampled experience following π — and by the law of large numbers the empirical average converges to the true expected return. The method assumes that every episode reaches a terminal state in finite time, that is, the task must be *episodic*; the return G_t is otherwise undefined.

- 4. Explain why Monte Carlo control estimates action values $Q(s, a)$ rather than state values $V(s)$.**

Improving a policy greedily requires comparing the actions available in each state. With a known transition model this comparison can be done from $V(s)$ by one-step lookahead, $\pi'(s) \in \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$. In the model-free setting the transition probabilities are unknown, so this lookahead is impossible. The action values $Q(s, a)$ circumvent the missing model because the greedy update $\pi'(s) \in \arg \max_a Q(s, a)$ depends only on quantities the agent already maintains from sampled returns.

- 5. State the exploration problem that arises when Monte Carlo control follows a deterministic policy and name the two standard remedies.**

A deterministic policy selects only one action in each state, so the returns of every other action are never observed and the estimates $Q(s, a)$ for those actions cannot improve. The first remedy is *exploring starts*, which begins each episode from a randomly chosen state-action pair so that every pair has a non-zero probability of being initiated. The second remedy is a *soft policy*, such as an ε -greedy policy, which assigns a positive probability to every action in every state and therefore continues to gather data on all actions without controlling the starting condition.

- 6. Numerical — ε -greedy action probabilities.** In a state s with $|\mathcal{A}(s)| = 5$ actions and $\varepsilon = 0.1$, the current action-value estimates are $Q(s, a_1) = 2.5$, $Q(s, a_2) = 4.0$, $Q(s, a_3) = 1.0$, $Q(s, a_4) = 3.5$, $Q(s, a_5) = 2.0$. (a) Identify the greedy action a^* . (b) Compute the probability assigned by the ε -greedy policy to a^* and to each non-greedy action and verify that the probabilities sum to one.

The greedy action is $a^* = a_2$ because $Q(s, a_2) = 4.0$ is the largest estimate. The exploration share spread over all actions is $\varepsilon/|\mathcal{A}(s)| = 0.1/5 = 0.02$, so the policy assigns probability

$$\pi(a^* | s) = 1 - \varepsilon + \frac{\varepsilon}{|\mathcal{A}(s)|} = 0.9 + 0.02 = 0.92,$$

and probability 0.02 to each of the four non-greedy actions. The total is $0.92 + 4 \times 0.02 = 1.0$, confirming a valid distribution.

Part B — Long Questions (8 marks each)

1. **Define the prediction and control problems for an episodic Markov decision process. Explain how Monte Carlo methods address each and why the model-free setting motivates their study.**

Definition. The *prediction* problem takes a fixed policy π and a Markov decision process and asks for the value function under that policy, that is, $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t = s]$ for every state s , or equivalently $q_\pi(s, a) = \mathbb{E}_\pi[G_t | S_t = s, A_t = a]$ for every state-action pair. The *control* problem asks for an optimal policy π_* such that $v_{\pi_*}(s) \geq v_\pi(s)$ for every s and every other policy π , together with the optimal value function v_* or q_* . Prediction is an evaluation task; control is an optimisation task.

The model-free setting. Dynamic programming solves both problems by repeated application of the Bellman equations, which require the transition probabilities $p(s', r | s, a)$. In most realistic problems — a robotic task, a card game such as blackjack, a recommendation engine — the transition model is either unknown or prohibitively large to store. The agent must therefore learn directly from *experience*: sequences of states, actions and rewards observed by interacting with the environment.

Monte Carlo approach to prediction. Monte Carlo prediction estimates $v_\pi(s)$ by averaging the returns observed from many sample episodes generated under π . The return from a visit to s is an unbiased sample of $v_\pi(s)$, and by the law of large numbers the empirical mean converges to the true expectation as the number of episodes grows. The method is purely sample-based: no transition probabilities are required and no current estimate is used to update another estimate.

Monte Carlo approach to control. For control the same averaging idea is applied to action values $q_\pi(s, a)$, because the greedy policy improvement $\pi'(s) \in \arg \max_a Q(s, a)$ needs no model. After each episode the estimates Q are updated using the observed returns and the policy is made (almost) greedy with respect to the new Q . This is an instance of *generalised policy iteration*: alternation of evaluation and improvement until the two are consistent.

Why study these methods.

- They learn directly from experience and require no model.
- They work with a simulator that produces sample transitions, even when the underlying probabilities are too complex to enumerate (as in blackjack).
- They can estimate the value of a single state of interest without sweeping every state, unlike dynamic programming.

Applications. Game playing (blackjack, board games), simulator-based control problems (vehicle dynamics, queueing systems) and policy evaluation in industrial recommendation systems are typical settings where the agent can generate complete episodes but has no analytic model of the dynamics.

2. Describe the first-visit Monte Carlo algorithm for policy evaluation. Provide the formal definition, the pseudocode, the incremental update, and discuss the convergence properties and the contrast with every-visit Monte Carlo.

Definition. First-visit Monte Carlo prediction estimates $v_\pi(s)$ by averaging, over many episodes generated under π , the returns observed from the *first* time s appears in each episode. Formally, for the k -th episode let $t_k(s)$ denote the first time step at which $S_{t_k(s)} = s$ (undefined if s does not appear). The estimate after K episodes is

$$V(s) = \frac{1}{N(s)} \sum_{k: s \text{ appears in episode } k} G_{t_k(s)},$$

where $N(s)$ is the number of episodes containing s and $G_{t_k(s)}$ is the return computed backwards from the terminal step using $G_t = R_{t+1} + \gamma G_{t+1}$.

Pseudocode.

First-Visit Monte Carlo Prediction

```

Input: policy pi to be evaluated; discount gamma
Initialize V(s) arbitrarily and Returns(s) = empty list, for all s

Loop for each episode:
    Generate an episode following pi:
        S_0, A_0, R_1, S_1, A_1, R_2, ..., S_{T-1}, A_{T-1}, R_T
    G = 0
    For t = T-1, T-2, ..., 0:
        G = gamma * G + R_{t+1}
        If S_t does NOT appear in S_0, S_1, ..., S_{t-1}:
            Append G to Returns(S_t)
            V(S_t) = mean( Returns(S_t) )

```

Incremental update. Storing every return is unnecessary because the sample average admits the constant-memory recurrence

$$V_{n+1}(s) = V_n(s) + \frac{1}{n+1} [G_{n+1} - V_n(s)],$$

where G_{n+1} is the $(n+1)$ -st first-visit return at s and n is the count of previous first-visit returns. This is the canonical

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \alpha [\text{Target} - \text{OldEstimate}]$$

template with step size $\alpha_{n+1} = 1/(n+1)$. Replacing the diminishing step size with a fixed constant α is preferred for *non-stationary* problems (the value function changes over time, as happens during control when the policy is being updated).

Convergence. Each first-visit return is an independent and identically distributed sample of $v_\pi(s)$; by the law of large numbers $V(s) \rightarrow v_\pi(s)$ as $N(s) \rightarrow \infty$. The estimation error decays like $1/\sqrt{N(s)}$, so quadrupling the number of episodes halves the typical error.

Comparison with every-visit Monte Carlo.

- Every-visit Monte Carlo drops the “first time” check and updates the estimate using the returns from *all* visits to s in every episode.
- First-visit returns within a single episode are independent of each other; every-visit returns within the same episode are correlated because they share suffix rewards.

- Both methods converge to $v_\pi(s)$ in the limit. First-visit has a slightly larger variance per visit but unbiased samples; every-visit has biased samples in the finite-sample regime but the bias vanishes in the limit.
- Every-visit is simpler to implement and extends more naturally to function approximation.

Strengths.

- Requires no model and no transition probabilities.
- Each state's estimate is independent of every other state's, so a particular state of interest can be evaluated without sweeping the rest.
- Does not bootstrap (no current estimate is used to update another), so the estimates are not biased by initialisation errors of nearby states.

Limitation. Updates can be made only at the end of an episode because the return G_t is unknown until the terminal step. The method therefore cannot be applied to continuing tasks and does not learn from partial trajectories.

3. Describe Monte Carlo control through generalised policy iteration. Discuss the role of action values, the policy improvement step, the assumptions made by the idealised scheme, and the exploration problem that motivates exploring starts.

Definition. Monte Carlo control seeks an optimal policy π_* for an episodic Markov decision process using only sampled episodes. It applies the *generalised policy iteration* schema — alternating evaluation and improvement — where evaluation is performed by averaging sampled returns and improvement is performed by making the policy greedy with respect to the latest action-value estimates.

Why action values rather than state values. In the model-free setting the greedy improvement $\pi'(s) \in \arg \max_a Q(s, a)$ is computed directly from quantities the agent maintains, whereas the lookahead based on V would require the unknown transition probabilities. Monte Carlo control therefore estimates $q_\pi(s, a)$ rather than $v_\pi(s)$, with a separate average kept for every state-action pair.

Idealised policy iteration with Monte Carlo. The scheme is

$$\pi_0 \xrightarrow{E} q_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} q_{\pi_1} \xrightarrow{I} \dots \xrightarrow{I} \pi_* \xrightarrow{E} q_*,$$

where the E step is a long phase of Monte Carlo evaluation under the current policy and the I step makes the policy greedy with respect to the resulting Q .

Policy improvement theorem. For the greedy policy $\pi_{k+1}(s) \in \arg \max_a q_{\pi_k}(s, a)$,

$$q_{\pi_k}(s, \pi_{k+1}(s)) = \max_a q_{\pi_k}(s, a) \geq q_{\pi_k}(s, \pi_k(s)) \geq v_{\pi_k}(s),$$

so $v_{\pi_{k+1}}(s) \geq v_{\pi_k}(s)$ for every state. Equality holds only when both policies are already optimal, which guarantees that repeated improvement either strictly increases the value function or has already reached the optimum.

Two idealised assumptions. The clean scheme above relies on two requirements that are not realistic in practice:

- Infinitely many episodes per evaluation phase, so that $Q \rightarrow q_{\pi_k}$ exactly before the next improvement.
- Exploring starts: every state-action pair must have a non-zero probability of being chosen to begin an episode, ensuring that all pairs are visited often enough to be estimated.

The first assumption is relaxed by performing evaluation and improvement *episode by episode* rather than waiting for full convergence — a Monte Carlo analogue of value iteration. The second assumption is addressed in two ways: the Monte Carlo Exploring Starts algorithm (which keeps the assumption literally) and on-policy soft-policy methods using an ϵ -greedy policy (which remove it).

The exploration problem. If π is deterministic, every state has only one action that is ever sampled. The returns of all the other actions in that state are never observed, so the estimates $Q(s, a)$ for those actions remain stuck at their initial values and the greedy improvement is biased toward the action that happened to be tried. Exploring starts is the simplest fix in simulation; soft policies are the necessary fix when the starting condition cannot be controlled.

Comparison with dynamic programming control.

- Policy iteration in dynamic programming sweeps every state during evaluation; Monte Carlo control samples only the states actually visited.
- Dynamic programming bootstraps: each state estimate uses the estimates of its successors. Monte Carlo does not.
- Dynamic programming requires a model; Monte Carlo control does not.

Applications. Monte Carlo control is the method of choice for problems where the rules are known but the underlying probabilities are awkward to enumerate (blackjack), and where a simulator is available so that exploring starts is possible.

4. Describe the Monte Carlo with Exploring Starts (Monte Carlo ES) algorithm in full. Provide the pseudocode, explain why exploring starts is needed, walk through one iteration, and discuss its convergence behaviour and limitations.

Definition. Monte Carlo ES is an on-policy control algorithm that uses generalised policy iteration with two adaptations:

- Evaluation and improvement are interleaved *after every episode*, rather than waiting for the action-value estimates to converge.
- Each episode is started from a randomly chosen state-action pair (S_0, A_0) with every pair receiving a non-zero probability of being selected as the initial pair. After A_0 is taken, the rest of the episode follows the current greedy policy π .

Pseudocode.

Monte Carlo ES (Exploring Starts)

```
Initialize, for all  $s$  in  $S$  and  $a$  in  $A(s)$ :
     $\pi(s)$           = arbitrary action
     $Q(s, a)$        = arbitrary real number
     $\text{Returns}(s,a)$  = empty list

Loop forever (for each episode):
    Choose  $S_0$  in  $S$ ,  $A_0$  in  $A(S_0)$  at random
        so that every pair has probability  $> 0$ 
    Generate the rest of the episode from  $(S_0, A_0)$ 
        following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, \dots, S_{T-1}, A_{T-1}, R_T$ 
     $G = 0$ 
    For  $t = T-1, T-2, \dots, 0$ :
         $G = \gamma * G + R_{t+1}$ 
        If the pair  $(S_t, A_t)$  does NOT appear
            in  $(S_0, A_0), \dots, (S_{t-1}, A_{t-1})$ :
```

```
Append G to Returns(S_t, A_t)
Q(S_t, A_t) = mean( Returns(S_t, A_t) )
pi(S_t) = argmax_a Q(S_t, a)
```

Why exploring starts is needed. A deterministic policy never selects more than one action per state. Without exploring starts the returns of all other actions are never observed, so the action-value estimates for those actions cannot improve and the greedy update is biased toward whichever action happened to be tried. Exploring starts guarantees that every (s, a) pair has a non-zero probability of beginning an episode and is therefore visited infinitely often in the long run, which is exactly the condition needed for the averages $Q(s, a)$ to approach $q_\pi(s, a)$ for every action.

Walk-through of one iteration.

- Sample a starting pair (S_0, A_0) at random so that every pair retains positive probability.
- Roll the episode forward from (S_0, A_0) using π until a terminal state is reached.
- Sweep the trajectory backwards, accumulating $G \leftarrow \gamma G + R_{t+1}$ at each step.
- On encountering the first occurrence of (S_t, A_t) in this episode, append G to the list of returns for that pair, update $Q(S_t, A_t)$ to the new mean, and make $\pi(S_t)$ greedy with respect to the updated $Q(S_t, \cdot)$.
- Evaluation and improvement are therefore *fused* into a single backward pass through the episode.

Convergence behaviour.

- Because all returns for each pair are averaged, Monte Carlo ES cannot settle on a suboptimal policy: a suboptimal stable point would force π to keep changing, which contradicts stability.
- Empirically, Monte Carlo ES recovers an optimal policy on benchmark problems such as blackjack, where after enough simulated games the learned policy matches the textbook basic strategy.
- A full theoretical proof of convergence in all cases remains an open question; partial results exist for restricted settings.

Strengths.

- Simple to implement: one episode roll-out followed by a backward sweep.
- Combines evaluation and improvement at no extra cost.
- Requires no exploration parameter to tune — exploration is supplied entirely by the starting distribution.

Limitations.

- Exploring starts is realistic only in simulation. In a real environment the agent typically cannot choose the starting state-action pair, so the assumption fails.
- Estimates can only be updated after an episode terminates, so the method is restricted to episodic tasks.
- For deterministic policies, sample efficiency depends on how often each pair is selected as the start; rare pairs receive few updates.

Bridge to on-policy soft methods. The on-policy first-visit Monte Carlo control algorithm replaces exploring starts with an ε -soft policy (every action has probability at least $\varepsilon/|\mathcal{A}(s)|$). Exploration then arises from the policy itself, the starting condition can be arbitrary, and the method becomes applicable to real environments at the cost of converging only to the best ε -soft policy rather than the true π_* .